Service Website — Project Blueprint (v2)

Device Support & Energy-Monitoring Service Company — Frontend UI Project

Document type: Requirements, design system, architecture, and working rules Status: Planning → Frontend UI build (no backend) Audience: The owner, Claude, and any other AI coding assistant

---

0. How to Use This Document

This file is the single source of truth for the project. Read it fully before changing anything.

Priority order when requirements conflict:

1. Honesty rules (Section 3) — never fake data, never fake form submission.
2. Scope rules (Section 2) — frontend only.
3. Working quality (Section 13) — everything shown must work.
4. Visual quality (Section 5) — premium, consistent design.
5. Everything else.

Three non-negotiables:

- **UI must look agency-grade.** Not a template, not a demo, not a college project.
- **Everything visible must work.** No dead links, no decorative buttons, no broken menus.
- **Nothing is invented.** No fake stats, reviews, certifications, contact details, or guarantees.

---

1. Project Overview

A professional website for a service company that handles electrical monitoring devices, energy-management equipment, and related technical solutions.

Website goal: Help a customer quickly understand which service fits their problem (faulty device, replacement, delivery, installation, support) and how to reach the company.

Current state: Frontend only. We are building the complete interface and page structure. Real submission, storage, notifications, and tracking come in a later phase.

Quality reference: https://wordpress.org/themes/neve/preview/ — use it only as a benchmark for polish (clean composition, clear navigation, good typography, spacing, responsive layout). Do not copy its branding, assets, or layout.

Questions every page must answer within seconds

- What services does the company provide?
- Which service matches my problem?
- Why should I contact this company?
- How does the process work?
- How do I contact the team or request a service?

Target users

- Customers with a faulty electrical-monitoring or IoT device
- Customers needing a new or replacement device delivered
- Customers needing installation or configuration help
- Customers needing maintenance or technical support
- Visitors who want to learn about the company and contact it

---

2. Scope

In scope

- Five complete pages with shared header, footer, and metadata
- Fully responsive layouts (desktop, tablet, mobile)
- Service cards, process section, FAQ accordion
- Contact form and Request-a-Service form with client-side validation
- Designed image placeholders with predictable filenames
- Basic SEO metadata and accessibility
- Organized PHP includes, one CSS file, one JS file
- README with local run instructions

Out of scope (do not build, do not hint at)

- Database / MySQL, backend processing, PHP form handlers that save or send data

- APIs or integrations
- Login, registration, roles, authentication
- Payments, real booking or scheduling, real-time tracking
- Email, SMS, WhatsApp, or push notifications
- Admin dashboard or CMS
- Fake customer data, statistics, reviews, or certifications

---

3. Honesty Rules (Highest Priority)

1. **Forms never pretend to submit.** On valid input, show a clearly labelled *demo confirmation* such as: "Your details look valid. This is a frontend preview — nothing has been sent or saved." Never write "Thank you, we received your request."
2. **No invented company facts.** No founding year, team size, client count, awards, certifications, response-time promises, warranty terms, or prices.
3. **No invented contact data.** Use obvious editable placeholders (see Section 14).
4. **Soft claims only.** Prefer "we can help you with…" over "guaranteed", "fastest", "#1", "24/7" unless the owner confirms.
5. **No fake social proof.** No testimonials, logos, or ratings.
6. **Document future needs** (backend, email, etc.) in the README, not in the visible UI.

---

4. Technology and Architecture

Stack

Layer	Choice	Notes
Templates	PHP (includes only)	For reuse, not for backend logic
Markup	Semantic HTML5	One `<h1>` per page
Styling	CSS3 with custom properties	Single `style.css`, organized by section
Behavior	Vanilla JavaScript	Single `main.js`, no libraries required
Optional	Bootstrap 5 grid/utilities	Only if it speeds work without limiting the design

Do not introduce Laravel, WordPress, React, Angular, Vue, Tailwind build steps, or any unnecessary framework.

Architecture principles

- **Single source of truth for content.** Services, nav links, contact placeholders, and FAQ items live in `includes/data.php` as PHP arrays and are rendered by loops. Changing a service name should require editing one place.
- **Central config.** `includes/config.php` holds `$site_name`, `$tagline`, contact placeholders, and base settings.
- **Per-page variables.** Each page sets `$page_title`, `$page_description`, `$current_page` before including the head and header.
- **Active nav** is determined from `$current_page`, not hard-coded per page.
- **Escape all dynamic output** with a small helper, e.g. `e($value)` wrapping `htmlspecialchars`.
- **Relative paths** work whether the site runs from a folder or the web root.
- **No inline styles or inline scripts** except tiny, justified cases.

Recommended structure

```
service-website/
├── index.php
├── services.php
├── about.php
├── contact.php
├── request-service.php
├── includes/
|   ├── config.php        (site name, tagline, contact placeholders)
|   ├── data.php          (services, nav, FAQ, process steps, why-choose
points)
|   ├── head.php          (doctype, meta, fonts, CSS link)
|   ├── header.php        (sticky header, nav, mobile menu)
|   ├── footer.php        (footer, JS include)
|   └── components.php    (small render helpers: image, icon, service card)
├── assets/
|   ├── css/style.css
|   ├── js/main.js
|   ├── images/           (photo1.jpg … photo10.jpg, README.md)
|   └── icons/            (README.md, optional SVG sprites)
├── README.md
└── .gitignore
```

Adjust only for a clear benefit. Do not add files just to look complex.

If an existing project folder is supplied

1. Inspect it before editing.
2. Follow its naming and structure.
3. Reuse existing components and styles.
4. Do not overwrite unrelated work or restructure without a clear reason.
5. Explain any structural change before making it.
6. Never commit or push to Git unless asked.

---

5. Design System

Visual quality is a top priority. The site should feel **premium, modern, trustworthy, clean, spacious, and technology/industrial-service oriented**, while staying easy for non-technical customers.

5.1 Color tokens

Define as CSS custom properties on `:root`.

Token	Value	Use
`--color-bg`	`#FFFFFF`	Main surface
`--color-bg-soft`	`#F5F7FA`	Alternate sections
`--color-bg-tint`	`#EEF3FD`	Subtle blue-tinted panels
`--color-ink`	`#0F1B33`	Headings, key text (deep navy)
`--color-text`	`#44506A`	Body text
`--color-muted`	`#6B7690`	Captions, helper text
`--color-border`	`#E3E8F1`	Card and input borders
`--color-primary`	`#497BE7`	Primary accent, links
`--color-primary-dark`	`#2F5FCB`	Hover/pressed, text on light backgrounds
`--color-mint`	`#5CE5BF`	Secondary accent (used sparingly)
`--color-error`	`#C62F3A`	Validation errors
`--color-success`	`#1E8E6A`	Valid-state indicators
`--gradient-accent`	`linear-gradient(135deg, #497BE7 0%, #5CE5BF 100%)`	Primary buttons, icon tiles, small highlights

Gradient discipline: buttons, icon tiles, thin accent lines, active states, small badges. Never a giant gradient hero or full-width gradient banners. White text on the gradient must keep readable contrast — if it does not, use dark navy text or the solid `--color-primary-dark`.

Dark navy sections (`--color-ink` background) are allowed once or twice per page (e.g., final CTA, footer) to create rhythm.

5.2 Typography

- **Headings:** a modern geometric sans such as *Plus Jakarta Sans* or *Manrope*.
- **Body:** *Inter* or a similar clean sans.
- Maximum two families. Load with `font-display: swap`, preconnect hints, and a full system-font fallback stack so the page looks right if fonts fail. Self-hosting in `assets/fonts/` is acceptable.
- Fluid sizes using `clamp()`:

Element	Size
H1 (hero)	`clamp(2.25rem, 5vw, 3.75rem)`, weight 700–800, line-height 1.1
H2 (section)	`clamp(1.75rem, 3.2vw, 2.5rem)`, weight 700, line-height 1.2
H3 (card)	1.25rem, weight 600
Body	1rem (16px minimum), line-height 1.65
Lead paragraph	1.125–1.25rem
Eyebrow label	0.8125rem, uppercase, letter-spacing 0.08em, primary color

- Paragraph max width about 65 characters (`max-width: 62ch`).
- Every section header follows one pattern: **eyebrow → H2 → short intro**.

5.3 Spacing, layout, and shape

- 8px spacing scale: `--space-1` (4px) through `--space-9` (96px).
- Container: max-width 1200px, side padding 20px on mobile, 32px on desktop.
- Section vertical padding: `clamp(64px, 9vw, 112px)`.
- Alternate section backgrounds (white / soft grey / tint) to create rhythm; avoid ten identical white blocks.

- Radius: 10px inputs and buttons, 16px cards, 24px large image frames. Do not mix many radii.
- Shadows: two levels only — soft resting shadow and a slightly deeper hover shadow, both low-opacity navy.
- Borders: 1px `--color-border`.

5.4 Buttons and links

Type	Style
Primary	Gradient background, white or navy text, 48px min height, subtle lift on hover
Secondary	White background, 1.5px primary border, primary text, tint on hover
Ghost / text link	Primary-dark text with arrow icon that nudges right on hover

All interactive elements need hover, `:focus-visible` (2–3px visible ring), and active states. Touch targets are at least 44×44px.

5.5 Icons

- Inline SVG or Bootstrap Icons — one style only (consistent stroke width).
- Place icons in rounded tiles (gradient or tint background).
- No emoji as interface icons.

5.6 Motion

- 150–250ms transitions for hover; 400–600ms for scroll reveals (fade + 16px rise).
- Use `IntersectionObserver` for reveals; reveal once, no repeating loops.
- Respect `prefers-reduced-motion: reduce` — disable transforms and reveals.
- No auto-playing video, no carousels, no parallax.

5.7 Avoid

Generic template look, excessive gradients, red/pink or neon "AI" visuals, many identical cards, overloaded hero, accidental empty areas, tiny or low-contrast text, inconsistent margins, fake statistics.

---

6. Shared Components

Header

- Sticky, white (slightly translucent with blur is fine), 1px bottom border or shadow that appears after scrolling.
- Left: logo placeholder (simple SVG mark + site name from config).
- Center/right: **Home, Services, About Us, Contact Us**.
- Right: **Request a Service** primary button.
- Active page link clearly indicated (color plus underline or pill).
- Height about 72px desktop, 64px mobile. All anchor targets use `scroll-margin-top` so headings never hide behind the header.

Mobile menu (below about 992px)

- Hamburger button with `aria-expanded` and `aria-controls`, 44px target.
- Panel slides or fades in under the header; includes all links and the CTA.
- Closes on: link click, Escape key, outside click, resize to desktop.
- Body scroll locked while open; focus moves into the menu and returns to the button on close.

Footer

- Dark navy background.
- Columns: brand + short description, Quick Links, Services (generated from `data.php`), Contact placeholders, social placeholders.
- Bottom bar: dynamic year via `date('Y')` and site-name placeholder.
- Social icons that have no real URL must not look clickable-but-dead: render them as disabled/muted with a title such as "Link coming soon", or omit them.

Page hero (inner pages)

- Soft tinted background, breadcrumb, H1, short intro. Compact — not a second homepage hero.

Image component

See Section 9. A single PHP helper renders every image with consistent wrapper, aspect ratio, alt text, and fallback.

---

7. Pages and Sections

Pages: `index.php`, `services.php`, `about.php`, `contact.php`, `request-service.php`.

Navigation from any page must work. Use real page links; homepage in-page anchors (e.g., `index.php#faq`) must also work when linked from other pages.

7.1 Homepage — `index.php` (most carefully designed page)

1. Hero

- Split layout: copy left, image frame right (`photo1.jpg`).
- Eyebrow, H1 (example: "Reliable Device Solutions. Expert Service. Zero Hassle."), supporting paragraph, two buttons (Request a Service / Explore Our Services).
- Three small value points with icons beneath the buttons (e.g., "Repair assistance", "Installation support", "Clear communication") — wording only, no numbers or guarantees.
- Image frame has a subtle decorative element (soft tinted shape or small floating info chip such as "Device support") — nothing that fakes data.
- Stacks on mobile: copy first, image second.

2. Service highlights strip

- Four compact items: Expert Device Support, Repair Assistance, Installation Support, Customer-Focused Service. Icon + title + one line.
- Lightweight style (no heavy cards) so it differs visually from the services section.

3. Our Services

- Section header (eyebrow, H2, intro).

- Four service cards from `data.php`, each with image (`photo2` – `photo5`), icon tile, title, short description, "Learn more" link to the matching `services.php` anchor.
- Equal heights, hover lift and image zoom (subtle), full-card focus state.
- Grid: 4 columns ≥1200px, 2 columns tablet, 1 column mobile.

4. Why Choose Us

- Editorial split layout: image (`photo6.jpg`) with offset frame on one side; content with 3–4 points on the other (technical assistance, clear communication, practical guidance, customer focus). Soft claims only.

5. How the Service Works

- Four steps: 01 Choose Your Service → 02 Contact Our Team → 03 Get Technical Assistance → 04 Resolve Your Device Issue.
- Horizontal with connecting line on desktop; vertical timeline on mobile.
- Informational only; do not imply tracking or automation.

6. Featured Service / Support CTA

- One featured service (repair & troubleshooting recommended) with `photo8.jpg`, short copy, and one CTA. A contained panel with rounded corners — not a full-bleed banner.

7. About Preview

- Short company introduction, image (`photo7.jpg`), link to `about.php`. Generic wording until facts are supplied.

8. FAQ

- Accordion with five questions (see Section 7.5). General answers only.

9. Final CTA

- Dark navy panel: "Need Help With Your Device?" with supporting text and two buttons (Request a Service, Contact Us).

10. Footer

- As in Section 6.

7.2 Services — `services.php`

- Inner-page hero with breadcrumb.

- Four detailed service blocks in alternating image/text layout; each has an anchor id (`#repair`, `#delivery`, `#installation`, `#support`), a short description, a "what this can include" list (generic), and a CTA that pre-selects the service in the request form via query string, e.g., `request-service.php?service=repair`.
- Process section (reuse the same component as homepage).
- Short FAQ (reuse component) and a closing CTA.

7.3 About — `about.php`

- Inner hero, company introduction, mission/values (generic, editable), service approach, reasons to get in touch, supporting images (`photo7.jpg`, `photo9.jpg`), CTA to Contact.
- No founding year, team size, customers, awards, or history.

7.4 Contact — `contact.php`

- Two-column layout: contact details card (phone, email, address, business hours placeholders) and contact form.
- Form fields: name, email, phone (optional), subject, message.
- Visible frontend-only notice near the form.
- No map unless a real address is supplied later.

7.5 Request a Service — `request-service.php`

- Clear heading, short explanation, and a small sidebar "What to include" (device model, issue, preferred contact).
- Fields: full name, phone, email, **service type** (select, options from `data.php`, pre-selected from `?service=`), device model, issue description, preferred contact method (radio: phone / email).
- Frontend-only notice visible above the submit button.
- Optional compact FAQ or tips panel below.

FAQ content (editable, general answers)

1. Which devices do you support?
2. How can I request a device repair?
3. Can I request a new device or replacement?
4. Do you provide installation assistance?

5. How can I contact technical support?

Answers must not state specific models, prices, timelines, or guarantees until confirmed.

---

8. Forms — Detailed Behavior

8.1 Validation rules

Field	Rule	Error message
Name	Required, 2+ characters	"Please enter your full name."
Email	Required, valid format	"Please enter a valid email address."
Phone	Required on request form; 7–15 digits, allows `+`, spaces, dashes, brackets	"Please enter a valid phone number."
Subject	Required (contact form)	"Please add a short subject."
Service type	Required (request form)	"Please choose a service."
Device model	Optional, max 80 characters	—
Message / issue	Required, 10–1000 characters	"Please describe your request in at least 10 characters."
Contact method	Required, one selected	"Please choose a contact method."

8.2 Behavior

- Use `novalidate` on the form and custom JS validation (consistent look across browsers).
- Validate each field on `blur`; re-validate on `input` once it has been marked invalid.
- On submit: if invalid, show error messages, set `aria-invalid`, and move focus to the first invalid field. Also show a short error summary at the top of the form.
- Errors use `role="alert"` or `aria-live="polite"` and are linked with `aria-describedby`.
- Show a live character counter on the message field.

- Visual states: default, focus, error (red border + icon + message), valid (subtle green check). Never rely on color alone.
- Required fields are marked with an asterisk and "(required)" available to screen readers.
- Correct `type`, `autocomplete`, and `inputmode` attributes (`email`, `tel`, `name`, and so on).

8.3 Valid submission (demo only)

- `preventDefault()` always. No `fetch`, no XHR, no storage, no `mailto:`.
- Replace or follow the form with a clearly labelled demo state: success-styled panel stating the details are valid and **nothing was sent or saved**.
- A "Reset form" button returns the form to its initial state.
- Mark integration points with comments like `<!-- BACKEND: replace demo handler with real endpoint -->` and keep field `name` attributes clean and meaningful (`full_name`, `email`, `phone`, `service_type`, `device_model`, `message`, `contact_method`) so a backend can be attached later.

---

9. Image System

The owner supplies real photos later. Do not download stock images, generate images, use remote URLs, or embed base64.

Filename mapping

File	Used for	Suggested ratio
`photo1.jpg`	Homepage hero	4:3 or 5:4
`photo2.jpg`	Device repair / troubleshooting	3:2
`photo3.jpg`	New device delivery / replacement	3:2
`photo4.jpg`	Installation / configuration	3:2
`photo5.jpg`	Maintenance / technical support	3:2
`photo6.jpg`	Why Choose Us	4:5 or 1:1
`photo7.jpg`	About Us	4:3

File	Used for	Suggested ratio
`photo8.jpg`	Featured service / CTA	16:10
`photo9.jpg`	Additional / supporting image	3:2
`photo10.jpg`	Optional extra	any

Path format: `assets/images/photoN.jpg`. Keep filenames and extensions identical in code and documentation. Document this table in `assets/images/README.md`, including recommended minimum widths (hero 1400px, cards 900px) and a tip to compress files (under about 250KB each where possible).

Rendering rules

- One helper function (e.g., `render_image($file, $alt, $ratio, $class)`) outputs a wrapper with a fixed `aspect-ratio`, `object-fit: cover`, `width` / `height` hints, `loading="lazy"` (not on the hero), and `decoding="async"`.
- **Designed fallback:** every wrapper has a neutral tinted background with a subtle line-pattern and a small SVG icon. If the file is missing, an `onerror` handler (in `main.js`, not inline) hides the broken `<img>` and reveals the fallback. The fallback must look obviously like a placeholder (for example, a small "Image: photo2.jpg" label in muted text) — never like a fake photograph.
- No layout shift when images are added later.
- Alt text must be meaningful and descriptive, and updated when real photos arrive.

---

10. JavaScript Behavior (`main.js`)

Only these interactions, each working reliably:

1. Mobile menu (open/close, Escape, outside click, focus handling, scroll lock).
2. Header shadow/state change on scroll.
3. Active nav styling (server-side first; JS only for in-page section highlighting if used).
4. Smooth scrolling for same-page anchors (disabled under reduced motion).
5. FAQ accordion — use `<button>` triggers with `aria-expanded` and `aria-controls`; Enter/Space toggle; smooth height animation; either single-open or multi-open, consistent across the site.
6. Form validation and demo states (Section 8).

7. Scroll reveal via `IntersectionObserver` (reduced-motion safe; content must remain visible if JS fails).
8. Image fallback handler.
9. Optional back-to-top button — only if the page is long enough to justify it.

Rules: wrap code in an IIFE or module pattern, guard against missing elements (pages share one script), no console errors, no global leaks, no heavy libraries.

---

11. Responsiveness

Test widths: 1440, 1024, 768, 375, and 320 px.

Breakpoints (guideline): 1200px, 992px (menu switch), 768px, 576px.

Must prevent: horizontal overflow, overlapping text, broken mobile nav, distorted images, awkward card heights, tiny tap targets, content hidden behind the sticky header, oversized mobile headings, forms wider than the viewport.

Use CSS grid and flexbox, `min-width: 0` on flex/grid children where needed, and `overflow-wrap: anywhere` for long words or emails.

---

12. Accessibility, SEO, and Performance

Accessibility

- Semantic landmarks: `header`, `nav`, `main`, `section`, `footer`.
- A "Skip to content" link as the first focusable element.
- Visible `:focus-visible` states everywhere.
- Text contrast at least WCAG AA (4.5:1 body, 3:1 large text).
- Labels tied to inputs via `for` / `id`; fieldsets and legends for radio groups.
- Menu and accordion fully keyboard operable.
- `prefers-reduced-motion` respected.
- Minimal, correct ARIA only.

SEO

- Unique `<title>` and meta description per page.
- Canonical-ready structure, Open Graph title/description placeholders.

- One `<h1>` per page with logical heading order.
- Descriptive link text and image alt text.
- Do not add structured data with invented business details.

Performance

- Lightweight: no unnecessary dependencies; CSS and JS each in a single file.
- Lazy-load below-the-fold images; avoid layout shift; limit font weights loaded.
- Prefer inline SVG for icons.

---

13. Working Quality — "Everything Must Work"

Before calling any task done, verify:

- [ ] All five pages load without PHP errors (`php -l` on every PHP file where tools allow).
- [ ] Every nav link, footer link, button, card link, and CTA goes somewhere real.
- [ ] `request-service.php?service=repair` (and the other three keys) pre-selects the right option.
- [ ] Homepage anchors work from other pages.
- [ ] Mobile menu opens, closes (link, Escape, outside click), and is keyboard usable.
- [ ] FAQ accordion works with mouse and keyboard.
- [ ] Both forms: invalid input shows correct errors, focus goes to the first error, valid input shows the demo state, reset works.
- [ ] No horizontal scroll at 320, 375, 768, 1024, 1440 px.
- [ ] Layout stays intact with **all images missing**, and again after real images are added.
- [ ] No browser console errors or warnings.
- [ ] Active nav state is correct on each page.
- [ ] Nothing in the UI claims a real submission, tracking, or unsupported guarantee.

Never claim something was tested if it was not actually tested. State clearly what was and was not verified.

---

14. Editable Placeholders

All placeholders live in `includes/config.php` or `includes/data.php`. Use clearly fake-looking values so nobody mistakes them for real data.

Item	Placeholder
Company name	`[Company Name]`
Tagline	`Reliable device solutions`
Phone	`+00 0000 000 000`
Email	`info@example.com`
Address	`[Business address]`
Business hours	`[Business hours]`
Social links	Disabled / "coming soon"

Starter service definitions (editable)

Key	Name	Image	Short description
`repair`	Device Repair & Troubleshooting	photo2.jpg	Help diagnosing and resolving issues with faulty monitoring devices.
`delivery`	New Device Delivery / Replacement	photo3.jpg	Support for getting a new or replacement device to you.
`installation`	Device Installation & Configuration	photo4.jpg	Guidance and assistance with installing and setting up your device.
`support`	Maintenance & Technical Support	photo5.jpg	Ongoing technical help to keep your systems running reliably.

These are examples, not confirmed commercial offers. Do not add guarantees, timelines, or prices.

---

15. Rules for Claude and Other AI Coding Assistants

1. Read this document before any change.
2. Identify the exact change requested.

3. Inspect relevant existing files before editing.
4. Make the smallest complete change that solves the request.
5. Reuse existing components, tokens, and helpers; do not duplicate styles or markup.
6. Preserve working pages and interactions; do not redesign unrelated sections.
7. Do not add backend, database, APIs, auth, or extra features without explicit approval.
8. Do not change the technology stack without asking.
9. Keep filenames, image mappings, and navigation consistent everywhere.
10. If something ambiguous materially affects the result, ask one concise question instead of inventing company facts.
11. After editing, verify using the checklist in Section 13 where tools exist.
12. Finish with a report: what changed, files touched, how to test, known limitations.
13. Never commit or push to Git unless explicitly requested.
14. For UI-only requests, never touch backend or server-side logic.

When the owner supplies real data (company name, logo, services, contacts, photos, brand colors, guidelines), replace placeholders with the supplied details and update the design tokens if brand colors differ.

---

16. Still Undecided (Do Not Assume)

- Final company name and logo
- Real phone, email, address, hours
- Approved service list and terms
- Supported device models
- Company history, team, certifications, achievements
- Official brand colors
- Real photographs and social profiles
- Whether a backend, email notifications, or booking will be added

---

17. Definition of Done — Version 1

Version 1 is complete when:

- All five pages exist, are visually complete, and share one consistent design system.

- The homepage makes a strong, premium first impression.
- Header, footer, and navigation are consistent and fully working.
- Services are clear, easy to edit from one data file, and linked to the request form.
- Image placeholders use the agreed filenames, look intentional, and never break the layout.
- Desktop, tablet, and mobile layouts are polished at all test widths.
- Mobile menu, FAQ accordion, and form validation work smoothly and accessibly.
- Forms validate in the browser and honestly show a demo-only state.
- No database, API, or backend logic exists.
- Code is organized, commented where helpful, and free of console and PHP errors.
- README explains setup, structure, how to edit content, and how to replace images.

---

18. README Must Include

1. Project summary and frontend-only status
2. How to run locally (for example, `php -S localhost:8000` in the project folder)
3. Folder structure
4. How to edit services, FAQ, nav, and contact placeholders
5. Image replacement guide and filename table
6. Design tokens and where to change colors/fonts
7. Testing checklist (Section 13)
8. **Future backend work** (list only): form handler, validation on server, spam protection, email/notification delivery, storage, admin tools

---

End of Project Blueprint